School of Computer Science and Engineering
(Computer Science & Engineering)
Faculty of Engineering & Technology
Jain Global Campus, Kanakapura Taluk - 562112
Ramanagara District, Karnataka, India

2025-2026
(VI Semester)

A Project Report on

"WanderChat: A Travel Companion"

Submitted in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

Submitted by

RUPALI - 23BTRCL072, PRABHAV JAIN- 23BTRCL067, ZAHID PICHEN - 23BTLCL006, M. JEEVAN SAI- 23BTRCL211 MOINAK MONDAL – 23BTRCL057 MANAS MISHRA -23BTRCL210

Under the guidance of

Dr. Archana Sasi
Assistant professor, CSE-AIML Department
JAIN (Deemed to-be university) Bengaluru.

Department of Computer Science and Engineering
School of Computer Science & Engineering
Faculty of Engineering & Technology
Jain Global Campus, Kanakapura Taluk - 562112
Ramanagara District, Karnataka, India

CERTIFICATE

This is to certify that the project work titled **"WanderChat: A Travel Companion"** is carried out by **RUPALI - 23BTRCL072, PRABHAV JAIN- 23BTRCL067, ZAHID PICHEN - 23BTLCL006, M. JEEVAN SAI- 23BTRCL211 MOINAK MONDAL – 23BTRCL057 MANAS MISHRA -23BTRCL210** bonafide student(s) of Bachelor of Technology at the School of Engineering & Technology, Faculty of Engineering & Technology, JAIN (Deemed-to-be University), Bangalore in partial fulfillment for the award of degree in Bachelor of Technology in Computer Science and Engineering, during the year **2025-2026**.

Dr. Archana Sasi
Assistant professor, CSE-AIML Department
JAIN (Deemed to-be university) Bengaluru.

Date:

Dr. V Chandrashekar
Computer Science and Engineering,
School of Computer Science & Engineering, Faculty of Engineering & Technology
JAIN (Deemed to-be University)
Date:

Dr. Geetha G
Director,
School of Computer Science & Engineering, Faculty of Engineering & Technology
JAIN (Deemed to-be University)
Date:

Name of the Examiner | Signature of Examiner

1.

2.

DECLARATION

We , **RUPALI - 23BTRCL072, PRABHAV JAIN- 23BTRCL067, ZAHID PICHEN - 23BTLCL006, M. JEEVAN SAI- 23BTRCL211 MOINAK MONDAL – 23BTRCL057 MANAS MISHRA -23BTRCL210** students of IV Semester B.Tech in **Computer Science and Engineering (Artificial Intelligence and Machine Learning)**, at School of Engineering & Technology, Faculty of Engineering & Technology, **JAIN(Deemed to-be University)**, hereby declare that the internship work titled **"WanderChat: A Travel Companion"** has been carried out by us and submitted in partial fulfilment for the award of degree in **Bachelor of Technology in Computer Science and Engineering** during the academic year **2025-2026**. Further, the matter presented in the work has not been submitted previously by anybody for the award of any degree or any diploma to any other University, to the best of our knowledge and faith.

Name: RUPALI USN :23BTRCL072	Signature
Name: PRABHAV JAIN USN :23BTRCL067	Signature
Name: ZAHID PICHEN USN :23BTRCL006	Signature
Name: MOINAK MONDAL USN:23BTRCL057	Signature
Name: MANAS MISHRA USN:23BTRCL210	Signature
Name: M. JEEVAN SAI USN: 23BTRCL211	Signature

Dr. Archana Sasi

Dr. Archana Sasi

ACKNOWLEDGEMENT

It is a great pleasure for me to acknowledge the assistance and support of a large number of individuals who have been responsible for the successful completion of this project work.

First, I take this opportunity to express my sincere gratitude to the Faculty of Engineering & Technology, JAIN (Deemed to-be University) for providing me with a great opportunity to pursue my Bachelor's Degree in this institution.

I am deeply thankful to several individuals whose invaluable contributions have made this project a reality. I wish to extend my heartfelt gratitude to ***Dr. Chandraj Roy Chand, Chancellor****, for his tireless commitment to fostering excellence in teaching and research at Jain (Deemed-to-be-University).*

It is a matter of immense pleasure to express my sincere thanks to ***Dr. V Chandrashekhar, Program Head of Computer Science and Engineering (Artificial Intelligence and Machine Learning)****, School of Computer Science & Engineering Faculty of Engineering & Technology for providing the right academic guidance that made my task possible. I would like to thank our guide* ***Dr. Archana Sasi, Assistant Professor, Jain University****, for sparing her valuable time to extend help in every step of my work, which paved the way for smooth progress and fruitful culmination of the project.*

I would like to thank our Project Coordinator ***Dr. Sankar K, PCL Coordinator****, and all the staff members of Computer Science and Engineering for their support.*

I am also grateful to my family and friends who provided me with every requirement throughout the course.

I would like to thank one and all who directly or indirectly helped me in completing the work successfully.

Signature of Student(s)

TABLE OF CONTENTS

List of Figures	Page No.
1. INTRODUCTION	**7-8**
1.1 Background & Motivation	7,8
2. LITERATURE SURVEY	**9-11**
a. Literature Review	9
b. Inferences Drawn from Literature Review	10,11
3. PROBLEM FORMULATION AND PROPOSED WORK	**12-19**
3.1 Introduction	12
3.2 Problem Statement	12-17
3.3 System Architecture /Model	18
3.4 Proposed Algorithms	18
3.5 Proposed Work	19
4. IMPLEMENTATION	**20-21**
4.1 Hardware Design and Implementation	20,21
5. RESULTS AND DISCUSSION	**22-23**
5.1 Result	22
5.2 Discussion	23
6. CONCLUSIONS AND FUTURE SCOPE	**24**
6.1 Conclusions	24
6.2 Future Scope	24
7. REFERENCES	**25**

ABSTRACT

The infectious illness known as Pneumonia is regularly a result of contamination because of a bacterium in the alveoli of the lungs. While an infected tissue of the lungs has an infection, it builds up pus in it. To find out if the patient has those illnesses, professionals perform bodily exams and diagnose their patients through Chest X-ray, ultrasound, or biopsy of lungs. Misdiagnosis, erroneous treatment, and if the disease is overlooked will result in the patient's lack of lifestyle. The progression of Deep learning contributes to aid in the decision-making procedure of specialists to diagnose sufferers with these illnesses. The look employs a bendy and efficient technique of deep learning of applying the model of CNN in predicting and detecting a patient unaffected and affected with the sickness using a chest X-ray photograph. The take a look at utilising an accrued dataset of 20,000 images using a 224x224 photograph decision with 32 batch length is applied to prove the overall performance of the CNN model being educated. The trained-version produced an accuracy charge of 95% at some point of the overall performance training. Based on the end result of the experiment carried out, the research study can detect and are expecting COVID-19, bacterial, and viral-pneumonia sicknesses based totally on chest X-ray photos.

LIST OF FIGURES

Sr. No.	Name of Figure	Page Number
3.1	Waterfall Model	10
4.1	Use Case diagram	13
4.2	Class Diagram	14
4.3	Sequence diagram	15
4.4	Activity Diagram	16
4.5	State Diagram	17
5.1	Examples from the dataset. (a) normal cases, (b) pneumonia case set	18
5.2	Details of proposed DL mode	19
6.1.1	Setting up Kaggle in Colab	20
6.1.2	Importing dataset from Kaggle in Colab	20
6.1.3	Extracting downloaded dataset in Colab	20
6.2.1	Defining parameters for Data Augmentation	21
6.2.2	Applying defined parameters to train, test and valid sets	21
6.2.3	Augmented Dataset Images	22
6.3	ResNet-50	23
6.4	AlexNet	23
6.5	VGG16	23
7.1	Augmented Dataset	24
7.2	Confusion Matrix	26
7.3	Predicted Images	27
7.4	Streamlit UI	28

LIST OF TABLES

Sr. No.	Name of Figure	Page Number
1	Literature Survey	4
2	Data Pre-Processing	19
3	Analysis of model performance for CNN model with 32 batch sizes	22

+

CONTENTS

CERTIFICATE i

ACKNOWLEDGEMENT ii

ABSTRACT iii

LIST OF FIGURES iv

LIST OF TABLES v

1. Introduction to Pneumonia Detection using CNN and VGG16 1

1.1 Introduction ……1

1.2 Motivation ……..……..1

1.3 Aim and Objectives of the work ……1

2. Literature Survey 3

3. Software Requirement Specification 5

3.1 Introduction ……...6

3.1.1 Project Purpose ……...6

3.2 Functional requirements ……7

3.3 Non-Functional requirements ……7

3.3.1 Safety Requirements ……...7

3.3.2 Security requirements …… 7

3.3.3 Software quality attributes …… 8

3.4 System Requirement ……. ……8

3.4.1 Software Requirement ……....8

3.4.2 Hardware Requirement ……...9

3.5 Analysis Models ……9

4. Design & Modelling of System 12

4.1. Use Case Diagram ……..…13

4.2. Class Diagram ……...….14

4.3. Sequence Diagram …….…15

4.4. Activity Diagram ……...16

4.5. State Diagram ……17

5. Algorithms and Methods 18

5.1. Data Acquisition ……18

5.2. Data Pre-processing ……....19

5.3. Proposed Network ……..19

6. Implementation and Testing 21

6.1 Data Acquisition ……...21

6.2 Data Augmentation……22

7. Evaluation 25

7.1 Image pre-processing ……25

7.2 Confusion Matrix ……..27

7.3 Visualizing some predicted images with percentage % ……28

7.4 User Interface ……29

8. Publication 30

9. Conclusion 31

10. References 32

CHAPTER 1

INTRODUCTION TO WANDERCHAT: AN AI TRAVEL COMPANION

1.1 INTRODUCTION

Travel planning, for decades, has been a fragmented, time-consuming, and often frustrating process. A typical traveler begins with search engines, navigates through dozens of travel blogs, watches videos, checks reviews on multiple platforms, and still ends up confused about what is genuinely worth visiting versus what is simply popular on the internet. This problem becomes even more pronounced when the traveler is searching for culturally rich destinations such as Kerala, where the true beauty of the region lies not only in famous backwaters and hill stations, but in small villages, seasonal festivals, local cuisine, community traditions, and experiences known primarily to locals.

Most traditional travel websites operate as static repositories of information. They provide lists of attractions, hotel options, and generic itineraries that are repeated across platforms. These websites lack contextual understanding of the traveler — who they are, why they are traveling, when they are visiting, and what type of experiences they value. As a result, two completely different travelers (for example, a honeymoon couple and a family with elderly parents) are often shown the exact same itinerary for Kerala. This one-size-fits-all approach fails to capture the personalized nature of travel planning.

With the rise of Large Language Models (LLMs), conversational AI systems have begun to appear in the travel domain. While these systems introduce an interactive way of querying travel information, they still suffer from critical limitations. Vanilla chatbots often rely purely on pre-trained knowledge, which may be outdated. They are also prone to hallucinating details, inventing attractions, or providing recommendations that are not grounded in real, local context. Such systems may sound intelligent but lack reliability, cultural depth, and factual accuracy.

WanderChat is designed to overcome these shortcomings by combining conversational intelligence with grounded knowledge retrieval and real-time information access. It is an AI-powered travel companion built specifically for Kerala tourism, designed not as a chatbot that merely answers questions, but as a digital local guide that understands travelers and plans journeys with them. WanderChat introduces a structured two-phase interaction model: Discovery and Planning. In the Discovery phase, the system learns about the traveler — their

preferences, travel group, budget, interests, and constraints. In the Planning phase, the system uses this information to generate personalized itineraries that are practical, culturally authentic, and up-to-date.

Technically, WanderChat leverages a Retrieval-Augmented Generation (RAG) pipeline that combines a curated Kerala travel knowledge base with real-time web data. This ensures that responses are not only intelligent but also factually grounded and temporally relevant. The platform is built using a modern technology stack: a Next.js frontend with a glassmorphism design aesthetic for an engaging user experience, a FastAPI backend for efficient API handling, and the Gemini 2.0 Flash model accessed through OpenRouter for conversational intelligence. The result is a seamless, interactive, and intelligent travel planning experience that bridges the gap between generic travel portals and personalized local guidance.

1.2 MOTIVATION BEHIND PROJECT TOPIC

Kerala, popularly known as "God's Own Country," is one of the most visited tourist destinations in India. It offers a diverse range of experiences — serene backwaters, misty hill stations, wildlife sanctuaries, pristine beaches, vibrant festivals, Ayurvedic wellness retreats, and deeply rooted cultural traditions. Despite this richness, most digital tourism platforms present Kerala in an overly simplified manner, highlighting only the most photographed and commercialized locations such as Munnar, Alleppey, and Kochi.

This approach creates two major problems. First, it results in overcrowding of a few tourist hotspots while many equally beautiful and culturally rich regions remain underexplored. Second, it deprives travelers of the opportunity to experience Kerala in an authentic and meaningful way. Many travelers leave Kerala feeling that they have "seen the places" but not truly experienced the culture.

Additionally, existing tourism portals tend to overwhelm users with raw information rather than curated guidance. Travelers are forced to manually filter through countless options, reviews, and recommendations to create an itinerary that suits their needs. This process requires significant time, effort, and prior knowledge. There is a noticeable gap between the generic

information provided by websites and the personalized advice one would receive from a knowledgeable local friend.

The motivation behind WanderChat is to bridge this gap. The project is driven by the vision of creating a system that behaves like a local companion who understands the traveler personally and suggests experiences accordingly. By considering factors such as travel purpose, group composition, season of travel, budget constraints, and personal interests, WanderChat aims to deliver deeply personalized travel plans rather than generic itineraries.

Another key motivation is aligned with the "Vocal for Local" philosophy. By surfacing lesser-known regions, small towns, local businesses, homestays, local eateries, and community-driven experiences, WanderChat aims to distribute tourism benefits more evenly across Kerala. This not only enhances the traveler's experience but also supports local economies and promotes sustainable tourism practices.

Furthermore, the project is motivated by the potential of modern AI technologies to transform how people plan travel. Instead of browsing static content, travelers can now engage in meaningful conversations with an intelligent system that understands context, retrieves relevant information, and generates actionable plans in real time.

1.3 AIM AND OBJECTIVES OF THE WORK

Aim:

The primary aim of this project is to design and develop an AI-powered conversational travel companion that delivers personalized, real-time, and culturally grounded itinerary recommendations for Kerala tourism. This is achieved by integrating a Retrieval-Augmented Generation pipeline with live web search capabilities, ensuring both depth of knowledge and freshness of information.

Objectives:

• To conduct an in-depth study of existing AI-driven travel recommendation systems and identify their limitations in terms of personalization, recency of information, and cultural authenticity.

• To design a two-phase interaction model — Discovery and Planning — that mirrors the natural way humans plan trips, starting from understanding preferences and ending with actionable itineraries.

• To create a curated Kerala travel knowledge base containing information about destinations, cultural experiences, local businesses, transportation options, and seasonal events.

• To implement a Retrieval-Augmented Generation (RAG) pipeline using sentence-transformer embeddings and a FAISS vector index to enable efficient semantic retrieval of relevant travel information.

• To integrate the Tavily Search API for real-time data retrieval regarding weather conditions, festivals, local events, and dynamic travel advisories.

• To develop a robust backend using FastAPI that handles API requests, manages the RAG pipeline, and communicates efficiently with the language model.

• To build a responsive, modern, and visually engaging frontend using Next.js, Tailwind CSS, and Framer Motion with a glassmorphism design aesthetic.

• To ensure that the system minimizes hallucinations by grounding responses in retrieved documents and live data sources.

• To evaluate the performance of the system based on personalization quality, response latency, factual accuracy, and freshness of surfaced information.

• To promote sustainable and local tourism by recommending lesser-known destinations and locally owned establishments.

• To create a scalable architecture that can be extended in the future to support other tourism destinations beyond Kerala.

Through these objectives, WanderChat aspires to redefine digital travel planning by transforming it from a search-driven process into an intelligent, conversational, and personalized experience.

CHAPTER 2

LITERATURE SURVEY

The WanderChat system lies at the intersection of three rapidly evolving domains: conversational AI applied to tourism, Retrieval-Augmented Generation (RAG) for factual grounding, and modern web application design for AI-first user experiences. Over the past few years, significant research and engineering progress in these areas has reshaped how intelligent assistants can be built for domain-specific applications. This chapter reviews the most relevant developments across these domains and explains how the insights drawn from them directly influenced the architectural and design decisions made in WanderChat

2.1 Conversational AI in Tourism

The earliest forms of travel chatbots were predominantly rule-based systems. These systems relied on predefined intents, keyword matching, and slot-filling mechanisms to answer user queries. While such systems were able to handle simple tasks like booking inquiries or FAQ-style responses, they lacked conversational flexibility. Users were required to phrase their queries in expected formats, and the systems were unable to handle multi-turn, context-aware dialogue.

The introduction of transformer-based Large Language Models (LLMs) revolutionized conversational AI by enabling systems to understand natural language more fluidly and maintain context over multiple conversational turns. These models could generate human-like responses, summarize information, and adapt to varied user inputs. As a result, many travel-related platforms began experimenting with LLM-powered assistants for itinerary suggestions and destination guidance.

However, studies and practical deployments have highlighted a critical challenge with using bare LLMs for domain-specific applications like tourism. LLMs rely heavily on parametric memory learned during training, which may be outdated, incomplete, or generalized. When asked about specific destinations, opening hours, pricing details, or local experiences, such

models often hallucinate plausible but incorrect information. This is particularly problematic in travel planning, where accuracy and reliability are essential.

Research in conversational AI therefore increasingly emphasizes the need for grounding LLM outputs in external knowledge sources rather than relying solely on the model's internal knowledge. This approach ensures factual correctness while preserving conversational fluency. WanderChat adopts this philosophy by using the LLM not as a standalone knowledge source, but as a reasoning engine conditioned on retrieved, verified context.

2.2 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation, introduced prominently in research by Lewis et al., proposes a hybrid architecture that combines information retrieval with generative modeling. In this framework, a retriever fetches relevant documents from a knowledge base at query time, and the generative model produces responses conditioned on those retrieved documents.

RAG has quickly become the standard architectural pattern for domain-specific AI assistants because it decouples knowledge from reasoning. The knowledge layer remains external, structured, and easily updatable without retraining the model. This allows systems to stay current, accurate, and adaptable to changes in domain information.

In tourism applications, this separation is particularly valuable. Travel information is dynamic: new attractions open, festivals change dates, transport routes get updated, and local businesses evolve. Embedding this information permanently inside a model is impractical. Instead, maintaining a curated, updatable knowledge base that the model can reference ensures that responses remain relevant and reliable.

WanderChat implements the RAG paradigm by maintaining a curated Kerala travel knowledge base indexed using FAISS. When a user asks a question or requests an itinerary, the system retrieves semantically relevant documents from this index. These documents are then supplied as context to the Gemini 2.0 Flash model, which generates responses grounded in this retrieved material. This architecture minimizes hallucinations and ensures cultural and factual accuracy.

2.3 Sentence Embeddings and Vector Search

Traditional information retrieval systems relied on keyword matching and bag-of-words representations, which often failed to capture semantic meaning. For example, a query for

"peaceful backwater stay" might not match a document describing a "serene houseboat experience" due to the absence of overlapping keywords.

Sentence-transformer models have addressed this limitation by producing dense vector embeddings that capture the semantic meaning of entire sentences. Models such as all-MiniLM-L6-v2 generate compact 384-dimensional vectors that represent the contextual meaning of text. These embeddings allow semantically similar sentences to be located in vector space even if they use different words.

To efficiently search over these embeddings, vector similarity search libraries such as FAISS (Facebook AI Similarity Search) are widely used. FAISS enables fast nearest-neighbour search over large collections of vectors, making it suitable for real-time applications.

The combination of MiniLM embeddings and a FAISS IndexFlatL2 index provides an excellent trade-off between retrieval accuracy and latency. This setup is particularly well-suited for interactive chat applications where responses must be generated quickly without compromising relevance.

WanderChat leverages this approach by embedding all documents in the Kerala travel knowledge base using MiniLM embeddings and indexing them in FAISS. When a query is received, the system computes its embedding and retrieves the most semantically relevant documents in milliseconds, ensuring both speed and contextual relevance.

2.4 Agentic Search and Real-Time Grounding

While a curated knowledge base provides depth and reliability, it cannot capture information that changes frequently, such as festival schedules, seasonal weather patterns, local events, or newly opened experiences. Relying solely on static documents would make the system outdated over time.

Recent advancements in agentic search APIs have enabled LLM-driven applications to perform real-time web searches in a structured and reliable manner. APIs such as Tavily expose web search capabilities in a way that is optimized for AI consumption, returning concise, citation-friendly results.

This concept of agentic search allows AI systems to augment their static knowledge with dynamic, real-world information at query time. For a travel assistant, this is invaluable. It allows

the system to answer questions like “What festivals are happening in Kerala next week?” or “Is the weather suitable for visiting Munnar this month?” with up-to-date information.

WanderChat integrates Tavily Search API into its pipeline to fetch live data whenever necessary. This ensures that the assistant remains temporally aware and can recommend experiences that are relevant to the traveler’s exact dates of visit.

2.5 Modern Frontend for AI Applications

The effectiveness of an AI assistant is not determined solely by its intelligence, but also by how users experience it. A poorly designed interface can make even the most advanced system feel cumbersome. Modern frontend frameworks have enabled developers to create highly responsive, visually appealing, and interactive interfaces with relatively low development overhead.

Next.js, built on React, provides server-side rendering, routing, and performance optimizations that are ideal for AI-powered web applications. Tailwind CSS enables rapid UI development with consistent styling, while Framer Motion introduces smooth animations that make interactions feel natural and engaging.

WanderChat adopts a glassmorphism design aesthetic — characterized by frosted, semi-transparent panels layered over vibrant Kerala-themed backgrounds. This design visually separates conversational content from the background while maintaining depth and modern appeal. Such visual choices are known to enhance user engagement and encourage longer, multi-turn interactions with AI systems.

2.6 Inferences Drawn from Literature Review

The literature and existing engineering practices collectively suggest several key principles for building an effective AI travel companion.

First, the system must ground its responses in a curated domain knowledge base to prevent hallucinations and ensure factual reliability.

Second, it must augment this static knowledge with live web search to handle time-sensitive and dynamic information.

Third, the system must personalize responses based on explicit user context such as travel style, group composition, preferences, and constraints.

Fourth, the system must deliver this intelligence through a polished, responsive interface that encourages natural, multi-turn conversation rather than one-off queries.

WanderChat is designed explicitly around these four principles. By combining RAG, vector search, agentic web search, and modern frontend design, the system aims to provide a travel planning experience that is intelligent, reliable, personalized, and engaging.

Sr. No.	Concept / Paper Theme	Relevance to WanderChat
1	Retrieval-Augmented Generation (Lewis et al.)	Core architectural pattern used to ground responses in Kerala travel documents.
2	Sentence-BERT / MiniLM embeddings	Used to convert text chunks and user queries into 384-dim vectors for semantic search.
3	FAISS: Efficient Similarity Search	Underlies the IndexFlatL2 vector store powering low-latency retrieval.
4	Agentic Search APIs (Tavily)	Enables live festival, weather, and hidden-gem lookups beyond the static dataset.
5	Modern AI Web UX (Next.js + Tailwind)	Informs the glassmorphic, mobile-first chat interface design.

Table 2.1 Literature Survey Summary

CHAPTER 3

SOFTWARE REQUIREMENT SPECIFICATION

3.1 INTRODUCTION

This Software Requirement Specification (SRS) provides a comprehensive and unambiguous description of the functional and non-functional behavior of the WanderChat system. The purpose of this chapter is to clearly define what the system is expected to accomplish, the operational constraints within which it must function, the characteristics of its users, and the technology stack that enables its implementation.

WanderChat is not merely a chatbot, but a structured AI-driven travel planning assistant designed specifically for Kerala tourism. Therefore, this SRS not only describes features in terms of inputs and outputs, but also explains how conversational flow, knowledge retrieval, real-time grounding, and user experience design collectively contribute to the system's objectives.

3.1.1 Project Purpose

The primary purpose of WanderChat is to provide travelers with an AI-driven conversational assistant that can understand their preferences and generate personalized, culturally rich, and up-to-date travel itineraries for Kerala. The system aims to replicate the experience of consulting a knowledgeable local guide who understands both the traveler and the destination.

Unlike generic travel portals, WanderChat focuses on personalization, contextual understanding, and real-time awareness. It highlights lesser-known destinations, local businesses, homestays, artisans, and regional food experiences in alignment with the "Vocal for Local" philosophy. The assistant is intended to reduce the time, effort, and confusion typically associated with travel planning by transforming it into a guided conversational experience.

3.1.2 Project Scope

- Provide a chat-based interface for travellers to discover Kerala destinations.
- Generate detailed, day-by-day itineraries based on user travel style, group type, and dates.
- Surface real-time festival and event information using web search.

- Recommend Vocal for Local options such as homestays, regional artisans, and small restaurants. The scope of WanderChat includes the following capabilities:
- Providing an interactive, chat-based interface where travelers can naturally express their travel preferences.
- Enabling users to discover Kerala destinations based on interests such as nature, culture, food, wellness, adventure, or relaxation.
- Generating structured, day-by-day travel itineraries tailored to travel dates, group composition (solo, couple, family, friends), travel style, and budget.
- Integrating real-time web search to surface festival schedules, seasonal events, and weather-related advisories relevant to the user's travel dates.
- Promoting "Vocal for Local" recommendations including homestays, small restaurants, local artisans, and hidden gems that are often absent from mainstream tourism websites.
- Maintaining conversational context so that the system can refine recommendations over multiple turns.

The scope does not include booking tickets, processing payments, or user authentication. The system focuses solely on intelligent travel planning and recommendations.

3.1.3 User Classes and Characteristics

The primary users of WanderChat are:

- Domestic tourists from within India planning a Kerala trip.
- International tourists seeking culturally authentic Kerala experiences.
- Students, families, couples, solo travelers, and groups of friends.

Users are expected to have basic familiarity with web browsers and chat interfaces. No technical knowledge is required. The application is designed to be intuitive and self-explanatory, guiding users through prompts and conversational cues.

The system assumes that users may not have prior knowledge of Kerala geography, local customs, or travel logistics. Therefore, responses are designed to be explanatory and user-friendly rather than assuming domain familiarity.

3.1.4 Assumptions and Dependencies

Assumptions:

1. The user has a stable internet connection.
2. The user is willing to share basic preferences such as travel dates and interests.
3. The OpenRouter and Tavily API services remain available.

Dependencies:

1. Availability of API keys for OpenRouter (Gemini 2.0 Flash) and Tavily.
2. Availability of the curated Kerala dataset placed in the /dataset directory.
3. Python 3.10+ with the libraries listed in Section 3.4.1.

3.2 FUNCTIONAL REQUIREMENTS

• **FR1** — The system shall accept user inputs describing travel style, group type, interests, and travel dates in natural language form.

• **FR2** — The system shall convert user queries into embeddings and retrieve semantically relevant documents from a FAISS vector index built over the Kerala knowledge base.

• **FR3** — The system shall query the Tavily API to obtain real-time information about festivals, events, weather conditions, and seasonal activities.

• **FR4** — The system shall operate in two modes:

- Discovery Mode: to understand user preferences and suggest destination types.
- Planning Mode: to generate a structured itinerary.

• **FR5** — In Planning Mode, the system shall generate a detailed, day-by-day itinerary that includes:

- Suggested destinations and activities
- Approximate budget estimates
- Vocal for Local recommendations
- Hidden Gems and offbeat experiences

• **FR6** — The system shall maintain conversational memory within a session to refine recommendations based on previous user inputs.

• **FR7** — The system shall display responses in a visually structured and readable format in the chat interface.

3.3 NON-FUNCTIONAL REQUIREMENTS

3.3.1 Performance Requirements

- The end-to-end response time for a typical query shall be under 5 seconds on a standard broadband connection.

- Vector similarity search over the FAISS index shall complete within 100 milliseconds for the operating dataset size.

3.3.2 Safety Requirements

• API keys and sensitive credentials shall be stored in environment variables and never exposed to the client side.

• All communication between frontend and backend shall use HTTPS in production deployment.

• The system shall avoid generating unsafe or misleading travel information by grounding outputs in retrieved context.

3.3.3 Security Requirements

The system does not require user accounts, so no personal credentials are persisted. Inputs are sanitized before being sent to the language model. CORS policies on the FastAPI backend restrict origins to the deployed frontend domain.

3.3.4 Software Quality Attributes

1. Functionality — produces relevant, grounded itinerary recommendations.
2. Performance — sub-5-second responses for typical interactions.
3. Security — no credential leakage, sanitized inputs, HTTPS in production.
4. Availability — designed for cloud deployment with health-checked endpoints.
5. Usability — chat-first UX with guided prompts and animated feedback.
6. Maintainability — clean separation between Knowledge Manager, retriever, and generator modules.

3.4 SYSTEM REQUIREMENTS

3.4.1 Software Requirements

- Operating System: Windows 10/11, macOS, or Linux
- Frontend: Next.js (React), Tailwind CSS v4, Framer Motion
- Backend: FastAPI (Python 3.10+), Uvicorn
- LLM Access: Google Gemini 2.0 Flash via OpenRouter
- Vector Database: FAISS (Facebook AI Similarity Search)
- Embedding Model: sentence-transformers all-MiniLM-L6-v2
- Real-time Search: Tavily Search API
- Document Processing: pdfplumber, python-docx
- Editor: Visual Studio Code

3.4.2 Hardware Requirements

- Processor: Intel Core i5 / AMD Ryzen 5 or higher
- RAM: 8 GB minimum (16 GB recommended for local index building)
- Storage: 256 GB SSD
- Network: Stable broadband connection

3.5 ANALYSIS MODELS (SDLC MODEL TO BE APPLIED)

WanderChat was developed using the Iterative SDLC model. Unlike the classic waterfall approach, the iterative model allowed the team to build a working baseline quickly — a simple LLM-only chatbot — and then progressively enhance it across well-defined cycles: adding the FAISS retriever, then the Tavily live search layer, then the two-phase Discovery and Planning interaction model, then the polished glassmorphic frontend. Each iteration was validated end-to-end before the next layer was added.

The five core SDLC activities were executed across these iterations:

1. Planning and requirement analysis — identifying the gap left by static tourism portals.

2. Designing the system architecture — separating retrieval, generation, and presentation layers.
3. Development — implementing the FastAPI backend, the RAG pipeline, and the Next.js frontend.
4. Testing — running representative travel queries and inspecting recall, latency, and output quality.
5. Deployment — packaging the backend and frontend for cloud hosting with environment-based configuration.

CHAPTER 4

DESIGN AND MODELLING OF SYSTEM

Unified Modelling Language (UML) is used to visually and conceptually describe the structure, behavior, and interaction patterns of the WanderChat system. Since WanderChat integrates multiple components — a conversational frontend, a FastAPI backend, a vector knowledge base, real-time search APIs, and an external LLM service — UML diagrams help in clearly understanding how these parts collaborate during a travel-planning conversation.

This chapter explains how users interact with the system, how internal modules depend on each other, and how data flows from user input to final itinerary generation. The diagrams represented in this chapter include the Use Case Diagram, Class Diagram, Sequence Diagram, Activity Diagram, and State Diagram.

4.1 Use Case Diagram

The Use Case Diagram represents the high-level interactions between external actors and the WanderChat system. It captures what the system does from the perspective of its users and supporting services.

The **primary actor** in the system is the **Traveller**. The traveller initiates conversations, provides travel preferences, asks for recommendations, and consumes the generated itineraries.

There are also **secondary actors** that support WanderChat's functionality:

- The OpenRouter LLM Service (Gemini 2.0 Flash), which generates conversational responses.
- The Tavily Search API, which provides real-time web information about festivals, weather, and events.
- The local FAISS Knowledge Base, which stores the curated Kerala travel dataset and supports semantic retrieval.

The key use cases identified in the system include:

- **Start Conversation** — The traveller opens the application and begins interacting with the assistant.
- **Provide Preferences** — The traveller shares travel dates, interests, group type, and travel style.
- **Discover Regions** — The system suggests diverse Kerala regions based on user inputs during Discovery mode.
- **Generate Itinerary** — The system produces a structured, day-by-day plan during Planning mode.
- **Fetch Live Events** — The system retrieves real-time festival and seasonal information via Tavily.
- **View Hidden Gems** — The system highlights lesser-known destinations and local experiences.
- **Refine Plan** — The traveller requests modifications to the generated itinerary.

This diagram establishes that WanderChat is not an isolated chatbot but an orchestrated system that interacts with multiple external services to fulfill user needs.

Fig 4.1 Use Case Diagram

4.2 Class Diagram

The Class Diagram models the internal structure of the WanderChat backend and shows how different classes collaborate to produce the final output.

The key classes in the system are:

KnowledgeManager
This class is responsible for preparing the knowledge base. It ingests documents from the /dataset directory, processes PDFs and DOCX files, splits them into manageable text chunks, generates embeddings using the sentence-transformer model, and builds the FAISS vector index. It is executed during the setup phase and whenever the dataset is updated.

Retriever
This class acts as a wrapper over FAISS. It exposes an API that takes a user query embedding

and returns the top-k most semantically relevant chunks from the knowledge base. It abstracts away the complexity of vector similarity search.

LiveSearchClient
This class encapsulates the integration with the Tavily Search API. It is responsible for sending structured queries to Tavily and receiving concise, relevant search results about festivals, events, and weather conditions.

ChatOrchestrator
This is the most critical class and acts as the entry point for user queries through the FastAPI endpoint. It coordinates the entire process:

- Accepts user input
- Calls the Retriever for local context
- Calls the LiveSearchClient for real-time information
- Constructs a prompt combining user data, retrieved context, and live results
- Sends the prompt to the Gemini 2.0 Flash model via OpenRouter
- Returns the generated response to the frontend

These classes are loosely coupled, ensuring modularity and maintainability. Each class has a single responsibility, making the system easier to extend and debug.

Fig 4.2 Class Diagram

4.3 Sequence Diagram

The Sequence Diagram shows the chronological order of interactions between system components during a typical Planning-mode conversation.

A standard interaction proceeds as follows:

1. **User Input**
   The traveller enters travel preferences such as dates, interests, and group type into the chat interface on the Next.js frontend.

2. **API Request to Backend**
   The frontend sends this input to the FastAPI backend endpoint handled by the ChatOrchestrator.

3. **Context Retrieval**
   The ChatOrchestrator converts the user query into an embedding using the all-MiniLM-L6-v2 model and performs a top-k similarity search on the FAISS index through the Retriever class.

4. **Parallel Live Search**
   At the same time, the LiveSearchClient sends a query to Tavily to fetch real-time data about festivals, weather, or seasonal events.

5. **Prompt Construction and LLM Call**
   The ChatOrchestrator composes a structured prompt that includes:
   - User preferences
   - Retrieved Kerala knowledge chunks
   - Live search results
     This prompt is sent to Gemini 2.0 Flash via OpenRouter.

6. **Response Generation**
   The LLM synthesizes all the information into a coherent, structured itinerary.

7. **Response Rendering**
   The backend streams the response back to the Next.js frontend, where it is rendered with animated message bubbles.

This sequence ensures that responses are grounded, up-to-date, and personalized.

Fig 4.3 Sequence Diagram

4.4 Activity Diagram

The Activity Diagram represents the internal control flow within the ChatOrchestrator class. It highlights the decision-making logic that determines how a query is processed.

The main decision point in the system is the **mode switch** between Discovery and Planning.

- If the user input is exploratory (e.g., “Where should I go in Kerala?”), the system enters **Discovery Mode**, where it retrieves diverse region suggestions and presents options.
- If the user input is intent-driven (e.g., “Plan my 5-day Kerala trip”), the system enters **Planning Mode**, which triggers the mandatory itinerary output template.

In Planning Mode, the activity flow ensures that the output includes:

- Day-by-day plan
- Budget estimate
- Vocal for Local recommendations
- Hidden Gems

This diagram clarifies that WanderChat is not producing free-form responses but follows a structured internal workflow.

Fig 4.4 Activity Diagram

4.5 State Diagram

The State Diagram models how WanderChat transitions between different conversational states during a user session.

The four primary states are:

Idle
The system waits for the user to start a conversation.

Discovery
Triggered when the user asks for suggestions or ideas without specifying detailed plans. The system suggests regions and experiences.

Planning
Triggered when the user requests a structured itinerary with dates and preferences.

Refinement

Triggered when the user asks for modifications to an existing plan, such as changing a day's activity or adjusting the budget.

Transitions occur based on user intent. For example:

- "Suggest places in Kerala" → Discovery
- "Plan my 5-day trip" → Planning
- "Make day 3 more adventurous" → Refinement

This state-based approach allows WanderChat to maintain conversational continuity and progressively refine travel plans rather than restarting the process each time.

Fig 4.5 State Diagram

CHAPTER 5

ALGORITHMS AND METHODS

This chapter explains how WanderChat is implemented at a technical level, focusing on how data is acquired, processed, indexed, retrieved, and finally used to generate grounded conversational responses. The system design follows a Retrieval-Augmented Generation (RAG) architecture enhanced with real-time web search and a structured interaction model. Each stage in the pipeline is carefully designed to balance semantic accuracy, response speed, and maintainability.

5.1 Data Acquisition and Ingestion

At the heart of WanderChat lies a curated Kerala travel knowledge base. Instead of relying on random internet sources at runtime, the system ships with an automated **Knowledge Manager** that processes a dedicated /dataset directory containing high-quality Kerala travel documents in PDF and DOCX formats. These documents include region descriptions, cultural notes, travel advisories, local experiences, food guides, and lesser-known attractions collected from reliable sources.

PDF files are parsed using the **pdfplumber** library. This library is chosen because it preserves the structural layout of documents, including tables, bullet points, and section formatting. Such structure is important because travel documents often present information in lists (e.g., “Top things to do,” “Local foods to try,” “Best time to visit”), and preserving this structure improves the semantic quality of the text used later for embedding.

DOCX files are parsed using **python-docx**, which allows extraction of headings, paragraphs, and formatting cues. During extraction, the Knowledge Manager performs a normalization step. This includes removing unnecessary whitespace, detecting headers and subheaders, preserving section boundaries, and ensuring that the text remains semantically coherent.

Once clean text is obtained, each document is split into smaller, manageable chunks. The chunk size is set to **600 words with a 100-word overlap** between consecutive chunks. This overlap is critical for semantic retrieval. Often, key information such as the name of a place or an activity may appear at the boundary between two chunks. Without overlap, such context could be lost,

reducing retrieval quality. Overlapping ensures continuity of meaning across chunks and improves the chances of retrieving complete, context-rich passages.

These processed chunks form the base units of the Kerala knowledge base used in semantic search.

5.2 Vectorization and Semantic Search

After chunking, each text chunk is converted into a numerical vector representation using the **sentence-transformers/all-MiniLM-L6-v2** embedding model. This model produces **384-dimensional dense vectors** that capture the semantic meaning of text rather than relying on keyword presence.

MiniLM is specifically chosen because:

- It provides strong semantic performance for sentence-level meaning.
- It is lightweight and runs efficiently on CPU, avoiding the need for GPU infrastructure.
- It offers a good balance between embedding quality and computational efficiency, which is ideal for real-time chat applications.

Once embeddings are generated, they are stored in a **FAISS IndexFlatL2** index. IndexFlatL2 performs an exact L2-distance search over vectors. Although approximate indices exist for very large datasets, the size of the Kerala knowledge base is small enough that exact search is both

feasible and fast. This eliminates approximation errors and ensures the most relevant chunks are always retrieved.

At query time, the user's message is embedded using the same MiniLM model. The system then performs a nearest-neighbour search against the FAISS index to retrieve the **top-k** most relevant chunks. Typical values of **k range from 4 to 8** depending on the complexity of the query.

The retrieved chunks are concatenated into a single structured context block. This block forms the factual grounding material that is later provided to the language model for response generation.

5.3 Live Web Search via Tavily

A static knowledge base, no matter how well curated, cannot capture information that changes frequently. Travel planning often depends on dynamic factors such as festival dates, seasonal weather conditions, road advisories, and newly opened attractions.

To address this limitation, WanderChat integrates the **Tavily Search API** for real-time web search. Whenever a user query includes date-sensitive or event-related elements, the system constructs a targeted search query and sends it to Tavily.

The query sent to Tavily is shaped using the user's stated travel dates, interests, and regions of focus. This ensures that returned results are temporally relevant and context-aware rather than generic search results.

The Tavily response provides concise summaries of relevant live information, which is then included alongside the retrieved FAISS context before sending everything to the LLM. This

combination ensures that WanderChat responses are both **deep (from the knowledge base)** and **fresh (from live search)**.

5.4 Prompt Composition and LLM Generation

The **ChatOrchestrator** is responsible for constructing the final prompt that is sent to the language model. This prompt is not a simple user message but a carefully structured input containing multiple components:

- A system instruction that defines WanderChat's persona, tone, and rules for operating in Discovery vs Planning mode.
- The retrieved context block from the FAISS knowledge base.
- The live search results from Tavily.
- The running conversation history to maintain context across turns.
- The current user message.

These elements are combined into a single prompt and sent to **Google Gemini 2.0 Flash** via **OpenRouter**. Gemini 2.0 Flash is selected because of its large context window, fast response time, and cost efficiency, which are critical for an interactive chat-based product.

The LLM then synthesizes all this information into a coherent, structured, and conversational response that is grounded in both static knowledge and live data.

5.5 Two-Phase Interaction Model

One of the most important design decisions in WanderChat is the separation of interaction into two distinct phases: **Discovery** and **Planning**.

The **Discovery phase** is designed for inspiration. At this stage, the user may not know exactly where they want to go. They describe who they are, what they enjoy, and when they plan to travel. Based on this, WanderChat suggests three to four diverse regions of Kerala that match the user's preferences. The focus here is on exploration rather than detail.

The **Planning phase** is designed for execution. Once the user selects a region or agrees to a high-level suggestion, WanderChat switches to Planning mode. In this phase, the system generates a structured, day-by-day itinerary that includes:

- Daily activities and timings
- Budget estimates
- Vocal for Local recommendations
- Hidden Gems and offbeat experiences

This separation is critical because it prevents the system from prematurely generating overly detailed plans before the user has chosen a direction. It mirrors how humans naturally plan trips — first exploring options, then committing to a detailed schedule.

CHAPTER 6

IMPLEMENTATION

6.1 Project Structure

The codebase is organized into a clean frontend/backend split:

```
wander-chat/
├── backend/
│   ├── app.py              # FastAPI entry point
│   ├── knowledge_manager.py # PDF/DOCX ingest + FAISS index
│   ├── retriever.py        # top-k semantic search
│   ├── live_search.py      # Tavily API client
│   └── orchestrator.py     # prompt assembly + Gemini call
├── dataset/                # PDF & DOCX Kerala travel documents
├── frontend/               # Next.js + Tailwind + Framer Motion
└── README.md
```

6.2 Backend Implementation

The backend of WanderChat is built using **FastAPI** and served using **Uvicorn**, forming a lightweight, high-performance API layer capable of handling real-time chat interactions.

Startup Workflow and Knowledge Initialization

When the backend server starts, an initialization routine is triggered by the Knowledge Manager. This routine performs the following steps automatically:

1. Scans the /dataset directory for all PDF and DOCX files containing curated Kerala travel content.
2. Extracts text from PDFs using **pdfplumber** and from DOCX files using **python-docx**.
3. Normalizes and cleans the extracted text while preserving headings and structural cues.
4. Splits the text into **600-word chunks with a 100-word overlap** to preserve semantic continuity.
5. Embeds each chunk using the **all-MiniLM-L6-v2** embedding model.
6. Stores the embeddings in a **FAISS** IndexFlatL2 index.

This FAISS index is **persisted to disk** after creation. As a result, subsequent server restarts do not require re-embedding the entire dataset, significantly reducing startup time and improving developer productivity.

The /chat Endpoint Workflow

The primary API endpoint exposed by the backend is /chat. This endpoint accepts:

- The latest user message
- The running conversation history

Upon receiving a request, the ChatOrchestrator performs the following operations:

1. Embeds the user message using MiniLM.
2. Queries the FAISS retriever to obtain top-k relevant knowledge chunks.

3. Simultaneously calls the Tavily live-search client to fetch real-time event or weather information.
4. Assembles a structured prompt containing:
    - System instructions
    - Retrieved context
    - Live search results
    - Conversation history
    - Current user input
5. Sends the prompt to **Gemini 2.0 Flash** through **OpenRouter**.
6. Streams the model's response back to the frontend.

Importance of Streaming

Streaming the LLM response is a crucial implementation choice. Instead of waiting for the entire itinerary to be generated, partial responses are sent to the frontend as soon as they are

produced. This makes the system feel responsive and conversational, especially when generating long, structured itineraries.

6.3 Frontend Implementation

The frontend of WanderChat is built as a modern web application using **Next.js**. Styling is handled by **Tailwind CSS** (v4), and animations are implemented using **Framer Motion**.

Visual Design and Glassmorphism

The interface adopts a **glassmorphism** design language. Message panels appear as frosted, semi-transparent cards layered over a vibrant Kerala-themed gradient background. This aesthetic serves two purposes:

- It visually separates conversation content from the background without feeling heavy.
- It gives the application a premium, modern feel aligned with consumer AI products.

Animation and Interaction Design

Animations are intentionally subtle to avoid distraction while maintaining a sense of liveliness:

- Messages fade and slide into view when received.
- A typing indicator pulses while the backend streams responses.
- Transitions between Discovery and Planning modes are visually distinct through slight layout and style changes.

The chat-first interface ensures that the user remains focused on the conversation rather than navigating complex menus.

6.4 Discovery Mode Behaviour

Discovery Mode is the default state when a conversation begins. The goal of this mode is **inspiration**, not detailed planning.

When the user starts chatting, the assistant asks one or two concise clarifying questions, typically about:

- Group type (solo, couple, family, friends)
- Travel style (relaxation, adventure, culture, food, wellness)

Based on the responses, the assistant retrieves relevant knowledge and suggests **three to four diverse Kerala regions** that match the user's interests.

Each suggestion follows a lightweight structure:

- A one-line engaging hook that captures the essence of the region.
- Three to four bullet points describing experiences, vibe, and highlights.

This approach prevents overwhelming the user with details and encourages exploration before commitment.

6.5 Planning Mode Behaviour

Planning Mode is triggered when the user expresses readiness to build a concrete itinerary. This may be through statements such as:

- "Let's plan five days in Wayanad"
- "Create a 4-day Kerala itinerary for my family"

Once in Planning Mode, the assistant switches to a **strict structured output format**. Every response in this mode must include the following mandatory sections:

Day-by-Day Itinerary

Each day is broken into:

- Morning activities
- Afternoon activities
- Evening activities

This time-based structure makes the plan practical and easy to follow.

Estimated Budget

A rough cost estimate is provided, broken down into categories such as:

- Stay
- Food
- Transport
- Activities

Vocal for Local Recommendations

This section highlights:

- Homestays
- Local eateries
- Regional artisans
- Small businesses

Hidden Gems

This section surfaces:

- Lesser-known attractions
- Offbeat experiences
- Places not commonly found on tourism portals

By enforcing this structure, WanderChat ensures that Planning Mode responses are actionable, culturally rich, and distinct from generic itineraries found online.

CHAPTER 7

RESULTS AND DISCUSSION

7.1 Personalization Quality

A core design objective of WanderChat is not merely to answer travel questions, but to **adapt its planning style to the traveller's persona**. To evaluate this, the system was tested against multiple representative traveller archetypes that frequently appear in real-world tourism scenarios: a solo backpacker operating on a tight daily budget, a family of four travelling with young children and elderly parents, a honeymooning couple seeking comfort and luxury, and a group of friends motivated primarily by adventure and social experiences.

For each profile, identical high-level prompts were used with only the persona variables changed. The resulting itineraries were then analyzed for pacing, choice of destinations, activity intensity, accommodation style, food recommendations, and budget structure.

The differences were not superficial — they were structural. For instance:

- The **family-with-children** persona consistently received itineraries with fewer daily transitions, more rest windows, beach and backwater locations over hill treks, child-safe activities, and recommendations for spacious homestays rather than compact hostels.
- The **solo backpacker** persona received plans with public transport suggestions, budget homestays, walking trails, street food spots, and flexible timing blocks that allowed for exploration without rigid scheduling.
- The **honeymoon couple** persona triggered recommendations for scenic stays, private experiences like houseboats and sunset viewpoints, premium dining options, and relaxed pacing focused on ambience rather than activity count.
- The **friends-adventure** persona consistently resulted in trek-heavy plans centered around Wayanad, Munnar, and Vagamon, with emphasis on viewpoints, waterfalls, jeep rides, and dense activity schedules.

This demonstrates that personalization emerges from the **interaction between persona-aware prompts, curated local data, and retrieval grounding**, rather than from generic LLM

behavior. The system does not simply swap destinations; it **reconstructs the entire travel philosophy** to match the traveller.

7.2 Real-Time Accuracy

One of the known limitations of standalone LLM systems is their knowledge cutoff. Travel planning, however, is highly sensitive to **time-dependent information** such as monsoon advisories, festival dates, seasonal closures, local events, and transport disruptions.

To evaluate WanderChat's ability to overcome this, targeted queries were issued around Kerala's major festival periods, including Onam and temple festival seasons. The system was asked questions whose answers would not exist in the LLM's static training data, such as current year festival dates and live advisories.

Because WanderChat integrates the Tavily live search layer in parallel with vector retrieval, the assistant was able to:

- Correctly surface current-year festival schedules
- Reference live weather and monsoon conditions
- Mention ongoing local events and advisories
- Adjust travel suggestions based on seasonal feasibility

When the same prompts were issued to a vanilla LLM without live search, the responses were either generic, outdated, or speculative. This validates the architectural decision to **augment**

static semantic memory with real-time web intelligence, ensuring factual freshness in contexts where timeliness is critical.

7.3 Latency

System responsiveness is a crucial factor in chat-based experiences. Users perceive long pauses as system failure, especially when requesting long-form outputs like multi-day itineraries.

WanderChat's architecture was intentionally optimized to minimize latency at every stage:

- The FAISS IndexFlatL2 runs locally in memory, avoiding network round trips to hosted vector databases.
- Retrieval operations across thousands of chunks consistently complete in under 100 milliseconds.
- Embedding with MiniLM runs efficiently on CPU without requiring GPU provisioning.
- The dominant latency contributor is the Gemini 2.0 Flash generation call.

To counteract this, response streaming is employed. Instead of waiting for the full itinerary to be generated, tokens are streamed to the frontend as soon as they are produced. This transforms

perceived latency into a "typing" experience, making the system feel significantly more responsive even when generating long answers.

Empirical observation during testing showed that users begin reading the itinerary within 1–2 seconds of submitting their query, despite total generation times sometimes exceeding 8–10 seconds for detailed plans.

7.4 Vocal for Local and Hidden Gems

A qualitative audit was conducted on dozens of generated itineraries to examine whether the system genuinely promotes local businesses and lesser-known experiences, or whether it defaults to globally visible tourist infrastructure.

The results showed a consistent pattern:

- Accommodation suggestions favored **small homestays**, eco-lodges, and family-run properties rather than international hotel chains.
- Food recommendations included **regional Kerala cuisine spots**, toddy shops, and local cafés instead of generic multi-cuisine restaurants.
- The Hidden Gems sections frequently mentioned:
    - Lesser-known viewpoints
    - Village walks and cultural interactions
    - Craft centers and spice farms
    - Waterfalls and trails not commonly found on mainstream travel portals

This behavior can be traced back to two design choices:

1. The curated /dataset contains rich local travel writing rather than scraped booking portals.
2. The prompt explicitly instructs the LLM to prioritize Vocal for Local and Hidden Gems.

Together, these ensure that WanderChat's outputs feel **locally grounded and culturally immersive**, differentiating it from standard itinerary generators.

7.5 Discussion

Several deliberate engineering trade-offs were made in WanderChat's design. While effective for the current scale, they reveal important considerations for future production deployment.

Index Choice — Exact vs Approximate Search
FAISS IndexFlatL2 provides exact nearest-neighbor results with no approximation error. This is ideal for small to medium datasets because of its simplicity and accuracy. However, its search time grows linearly with dataset size. If the knowledge base expands by an order of magnitude, the system would need to migrate to approximate indices such as IVF (Inverted File Index) or HNSW (Hierarchical Navigable Small World) to maintain low latency.

Embedding Model Size vs Speed
MiniLM provides an excellent speed-to-quality ratio and runs on CPU, keeping infrastructure lightweight. However, larger embedding models (e.g., MPNet-based sentence transformers) would likely improve semantic recall for subtle or complex queries. The trade-off would be increased embedding time and potentially higher memory usage.

External Dependencies and Availability Risk
The reliance on OpenRouter for Gemini access and Tavily for live search introduces third-party availability risks. In a production-grade system, this would be mitigated through:

- Response caching for frequent queries
- Graceful fallbacks when live search fails
- Redundant model providers where possible
- Retry and timeout strategies

Prompt Length vs Context Window
As conversation history grows, prompt size increases. Although Gemini supports large contexts, careful prompt pruning and summarization would be necessary for very long sessions to avoid unnecessary token usage and latency.

CHAPTER 8

PUBLICATION

The WanderChat project has been documented as a self-contained engineering report for academic submission. The full implementation, including the FastAPI backend, the Next.js frontend, the Knowledge Manager, and the curated Kerala dataset, is publicly available as an open-source repository at https://github.com/prabhavjain2004/wander-chat. The team intends to extend the work into a formal publication describing the two-phase Discovery and Planning interaction model and its impact on personalization quality.

CHAPTER 9

CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

WanderChat demonstrates that a thoughtful combination of Retrieval-Augmented Generation, agentic web search, and a two-phase conversational design can move AI travel assistants beyond the generic chatbot baseline. By grounding Gemini 2.0 Flash's output in a curated Kerala knowledge base via FAISS, augmenting it with real-time results from Tavily, and presenting it through a polished glassmorphic Next.js interface, the system delivers personalized itineraries that respect the user's group type, travel style, and dates while consistently highlighting Vocal for Local options and Hidden Gems.

The project also confirms that careful prompt design and a well-defined Discovery-versus-Planning split materially improve the conversational experience: the assistant feels less like a search engine and more like a knowledgeable local friend.

9.2 Future Scope

- Extend WanderChat to additional Indian states beyond Kerala by ingesting state-specific datasets.
- Add multilingual support — Malayalam, Tamil, Hindi — to broaden domestic accessibility.
- Replace IndexFlatL2 with an approximate HNSW or IVF index as the knowledge base grows.
- Introduce booking integrations with homestay and transport partners to close the loop from planning to reservation.
- Add personalization memory across sessions so returning users get progressively better recommendations.
- Build a mobile-native client that uses on-device inference for offline-friendly itinerary browsing.

CHAPTER 10

REFERENCES

[1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," Advances in Neural Information Processing Systems (NeurIPS), 2020.

[2] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," Proceedings of EMNLP-IJCNLP, 2019.

[3] J. Johnson, M. Douze, and H. Jegou, "Billion-scale similarity search with GPUs," IEEE Transactions on Big Data, 2019. (FAISS library.)

[4] A. Vaswani et al., "Attention Is All You Need," Advances in Neural Information Processing Systems (NeurIPS), 2017.

[5] Google DeepMind, "Gemini 2.0: Our most capable AI model yet," Technical Blog, 2024. https://deepmind.google

[6] OpenRouter, "Unified API for Large Language Models," Documentation, 2024. https://openrouter.ai

[7] Tavily, "Tavily Search API for Agentic AI Applications," Documentation, 2024. https://tavily.com

[8] Vercel, "Next.js — The React Framework for the Web," Documentation, 2024. https://nextjs.org

[9] Tailwind Labs, "Tailwind CSS — A utility-first CSS framework," Documentation, 2024. https://tailwindcss.com

[10] Framer, "Framer Motion — A production-ready motion library for React," Documentation, 2024. https://www.framer.com/motion/

[11] S. Ramirez, "FastAPI — Modern, fast (high-performance) web framework for building APIs with Python," Documentation, 2024. https://fastapi.tiangolo.com

[12] J. Singer-Vine, "pdfplumber — Plumb a PDF for detailed information about each text character, rectangle, and line," GitHub, 2024. https://github.com/jsvine/pdfplumber

[13] python-docx contributors, "python-docx — Python library for creating and updating Microsoft Word files," Documentation, 2024. https://python-docx.readthedocs.io

[14] Hugging Face, "sentence-transformers/all-MiniLM-L6-v2 Model Card," 2024. https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

[15] Kerala Tourism, Government of Kerala, Official Tourism Portal, 2024. https://www.keralatourism.org

[16] WanderChat Source Code, "wander-chat repository," GitHub, 2026. https://github.com/prabhavjain2004/wander-chat